Software Architecture Specification

Architectural Requirements

1. Architectural Design Strategy

We worked closely with the client to understand what they needed from the system. Our main focus was on making it fast, reliable and easy to maintain.

Why we did it this way:

- Client First - put their needs at the center of every decision.
- Saves Time and Money - proper planning avoids rework.
- Better Technical Choices - knowing requirements helped pick the right tools.

2. Architectural Patterns

2.1 Layered Architecture

We split the system into four layers, each with its own job. This keeps things organized and makes it easier to change stuff without breaking everything.

The Layers:

- **Presentation Layer (Frontend):** React application that users interact with. Communicates with the backend via REST API. Never touches the database directly.
- **Business Logic Layer (Backend):** Express API that handles all business rules. Processes telemetry, calculates safety scores, manages trips.
- **Data Access Layer:** Abstracts all database operations. Keeps database logic separate from business logic.
- **Database Layer:** PostgreSQL with TimescaleDB extension. Stores telemetry, safety scores, trips, and user data.

Why we chose it: Clear boundaries between concerns. When we change safety score calculations, we only touch the business layer. When we change the database, we only touch the data access layer.

2.2 Event-Driven Architecture

Vehicles send telemetry to AWS Kinesis, which triggers Lambda functions to process and store the data.

How it works:

- Vehicle sends telemetry to Kinesis.
- Kinesis triggers Lambda.
- Lambda processes and stores data.
- Dashboard updates automatically.
- Handles large amounts of data well.
- Keeps vehicles decoupled from the backend.
- Easy to extend later.
- No polling needed - events are pushed.

2.3 Medallion Architecture

Data is stored in three layers using TimescaleDB:

- **Bronze (**raw_telemetry**):** Raw data kept as-is. Never modified. Good for debugging.
- **Silver (**clean_telemetry**,** vehicle_events**):** Cleaned and organized. GPS parsed, speeds validated.
- **Gold (**current_vehicle_position**,** driver_daily_safety_scores**):** Ready for fast dashboard queries. Pre-computed so queries are fast.

Why we chose it: We keep the raw data if something goes wrong. The gold layer makes dashboard queries instant because the heavy work is already done.

2.4 Client-Server Architecture

React frontend talks to an Express API, which queries the database.

- Frontend and backend worked on independently.
- Clear separation of responsibilities.

3. Design Patterns

3.1 Singleton Pattern

What it is: Ensures only one instance of a class exists and provides a global point of access to it.

Where we use it: Database connection pool. We create one pool and reuse it everywhere.

```
const { Pool } = require('pg');
const pool = new Pool({
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT || '5432'),
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,});
module.exports = { pool };
```

Why we used it:

- Saves resources by reusing connections.
- Prevents connection leaks.

- Manages connection limits efficiently.

3.2 Factory Pattern

What it is: Creates objects without exposing the creation logic to the client.

Where we use it: API response formatting. The success() and error() functions create consistent response objects.

```
function success(res, data, statusCode = 200) {
  return res.status(statusCode).json({
    success: true,
    data,
    timestamp: new Date().toISOString()
  });
}

function error(res, message, statusCode = 500) {
  return res.status(statusCode).json({
    success: false,
    error: message,
    timestamp: new Date().toISOString()
  });
}
module.exports = { success, error };
```

Why we used it:

- Consistent API response format across all endpoints.
- Easy to change response structure in one place.

3.3 Observer Pattern

What it is: Defines a one-to-many dependency where when one object changes state, all its dependents are notified.

Where we use it: Kinesis to Lambda integration. When new telemetry arrives in Kinesis, it automatically triggers the Lambda function. Alert handling stays separate from detection.

```
exports.handler = async (event) => {
  for (const record of event.Records) {
    const data = JSON.parse(Buffer.from(record.kinesis.data, 'base64'));
    await processTelemetry(data);
  }
};
```

Why we used it:

- Decouples data ingestion from processing.
- No polling needed - events are pushed.
- Handles variable telemetry loads.

3.4 Chain of Responsibility

What it is: Passes a request along a chain of handlers. Each handler decides to process the request or pass it to the next handler.

Where we use it: API middleware chain. Each request passes through multiple middleware handlers.

app.use(cors());

```
app.use(helmet());
app.use(limiter());
app.use('/api/vehicles', authenticate, requireRole(['admin', 'fleet_manager', 'viewer']), vehicleRoutes);
```

```
async function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) {
    return error(res, 'No token provided', 401);
  }
  req.user = decodedUser;
  next();
}
```

Why we used it:

- Clean separation of cross-cutting concerns.
- Easy to add new middleware without changing existing code.

3.5 Strategy Pattern

What it is: Defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime.

Where we use it: Safety score calculation. Different event types have different penalty strategies.

```
SELECT COALESCE(SUM(
    CASE
      WHEN event_detail = 'harsh_braking' THEN 10
      WHEN event_detail = 'harsh_acceleration' THEN 5
      WHEN event_detail = 'harsh_cornering' THEN 3
      WHEN event_category = 'crash_detection' THEN 50
      ELSE 0
    END
), 0) INTO total_penalty
FROM vehicle_events
```

```
WHERE vehicle_id = v_record.vehicle_id
  AND DATE(time) = yesterday;
 score := GREATEST(100 - total_penalty, 0);
```

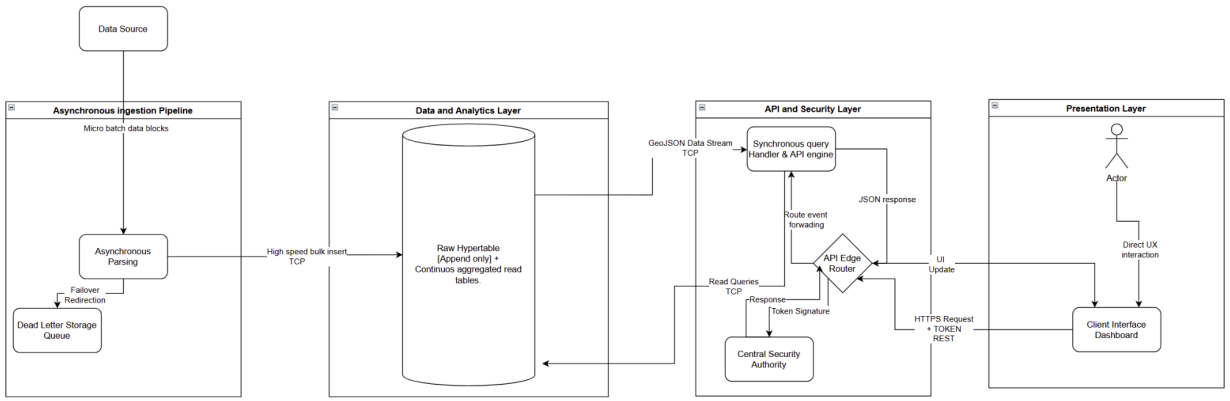
Why we used it:

- Easy to add new event types with different penalties.
- Penalty values can be changed without rewriting logic.

3.6 Memento Pattern

Where we use it: Trip replay and trip history. Captures and restores a vehicle's past state without exposing internal details, so a trip can be "replayed" step by step.

4. Architectural Diagram



5. Quality Requirements

5.1 Performance

- **API response time:** Under 2s - gold layer pre-aggregates; DB indexes; connection pooling.
- **Telemetry processing:** Under 2s - Lambda auto-scales; TimescaleDB hypertables.
- **Dashboard load:** Under 5s - React + Zustand; API Gateway caching.
- **Map updates:** Under 10s - Kinesis stream; continuous aggregate.

5.2 Scalability

- **Vehicles:** 15 now, scalable to 50 - Lambda + Kinesis shards.
- **Records/min:** 48 now, scalable to 160 - TimescaleDB hypertables.
- **Users:** 50 now, scalable to 500 - serverless architecture.
- **DB connections:** 100 - PgBouncer.

5.3 Security

- **Data at rest:** AES-256 via AWS KMS.
- **Data in transit:** TLS 1.2+ via API Gateway and HTTPS.
- **Authentication:** JWT tokens via AWS Cognito.
- **Authorization:** Role-based middleware.

- **Rate limiting:** 100 requests per 15 minutes via express-rate-limit.

5.4 Reliability

- **Uptime:** 99.5% - serverless AWS.
- **Data durability:** 11 nines - S3 storage.
- **Failure recovery:** Within 5 minutes - CloudWatch + auto-retry.
- **Data loss:** Kinesis at-least-once + dead letter queues.

5.5 Maintainability

- **Test coverage:** 80%+ - Jest + Codecov.
- **Code quality:** Zero ESLint errors - ESLint.
- **Deployment:** Under 2 hours - GitHub Actions CI/CD.
- **Local setup:** Under 30 minutes - Docker Compose.

5.6 Usability

- **Learnability:** Under 5 minutes - simple UI, clear labels.
- **Error messages:** Clear and helpful.

6. Constraints

Technical

- Cloud provider: AWS only.
- No Personally Identifiable Information stored: we do not store anything that can identify a person - we only keep vehicle data.
- Must handle 15 vehicles at 5-10 second intervals.
- Browser support: latest Chrome, Safari, Edge.

Development

- Minimum 80% test coverage.
- Minimum 4 PR approvals before merge.
- No direct pushes - all changes via PR to develop.
- Docker Compose for local dev.

Business

- 4 fixed demo milestones (May, July, September, October).
- Must meet all COS301 capstone specifications.
- FuseIT branding on all deliverables.
- Code must be publicly accessible.

Budget

- AWS cost limit: R5000 - R15 000.
- CloudWatch budget alerts.
- Serverless-first to minimize cost.

7. Technology Requirements

Frontend

React.js
Purpose: UI framework for dashboards and maps.

Why we chose it:

- Mapbox GL JS has better React integration than Vue or Angular.
- Recharts works natively with React components.
- Our team has experience with React, which speeds up development.

What we considered:

- **Vue.js:** Simpler to learn but Mapbox integration is not as smooth.
- **Angular:** More opinionated and heavier, would slow us down.

Vite
Purpose: Build tool and development server.

Why we chose it:

- Faster hot reload than Webpack, which saves time during development.
- Less configuration needed than Webpack or CRA.

What we considered:

- **Webpack:** Slower and more complex to configure.
- **Create React App (CRA):** Slower and harder to customize.

Tailwind CSS
Purpose: Styling and UI components.

Why we chose it:

- Matches our brand colours easily.
- Keeps styles consistent across all components.
- No need to write custom CSS for every component.

What we considered:

- **Plain CSS:** Harder to keep consistent across the team.
- **Sass:** More setup and configuration required.

Mapbox GL JS
Purpose: Interactive vehicle tracking maps.

Why we chose it:

- Handles 15+ vehicle markers updating every 5-10 seconds without lag.
- Supports custom vehicle icons and colour-coded statuses.
- Good documentation and examples.

What we considered:

- **Leaflet:** Lighter but doesn't handle real-time updates as well.
- **Google Maps API:** More expensive for commercial use.

Recharts
Purpose: Charts and graphs for analytics.

Why we chose it:

- Built specifically for React, so it integrates seamlessly.
- Supports all chart types we need (line, bar, donut).
- Lightweight and responsive.

What we considered:

- **Chart.js:** Not React-native, requires wrapper libraries.

Zustand
Purpose: State management.

Why we chose it:

- Lightweight and simple to use for auth state and vehicle data.
- Less boilerplate than Redux.
- Works well with React.

What we considered:

- **Redux:** More powerful but adds unnecessary complexity for our needs.
- **Context API:** Can cause unnecessary re-renders.

Backend

Node.js
Purpose: JavaScript runtime.

Why we chose it:

- Allows us to use JavaScript across the entire stack.
- Handles I/O operations well for API requests and database queries.
- Large ecosystem for backend libraries.

What we considered:

- **Python:** Better for data science but slower for API requests.
- **Go:** Faster but would require learning a new language.

Express.js
Purpose: REST API framework.

Why we chose it:

- Simple and flexible enough for our needs.
- Good middleware support for JWT validation, rate limiting, and CORS.
- Well documented and widely used.

What we considered:

- **Fastify:** Faster but less mature ecosystem.
- **NestJS:** More structured but adds unnecessary complexity.

JWT
Purpose: Authentication tokens.

Why we chose it:

- Stateless, which works well with serverless architecture.
- Integrates with Cognito.
- Standard format that works with any OAuth2 client.

What we considered:

- **Session-based auth:** Requires server-side storage, not suitable for serverless.

Helmet
Purpose: Security headers.

Why we chose it:

- Adds security headers with minimal configuration.
- Protects against common web vulnerabilities.

What we considered:

- **Custom headers:** Would take more time to implement correctly.

express-rate-limit
Purpose: API rate limiting.

Why we chose it:

- Prevents brute-force attacks.
- Easy to configure.
- Works with Express.

What we considered:

- **API Gateway throttling:** Already used, but this adds an extra layer.

pgBouncer
Purpose: PostgreSQL client.

Why we chose it:

- Connection pooling reduces latency for repeated queries.
- Works with TimescaleDB extensions.
- Lightweight and direct SQL control.

What we considered:

- **Sequelize:** Adds ORM overhead for our use case.
- **TypeORM:** More complex and slower.

serverless-http
Purpose: Express to Lambda wrapper.

Why we chose it:

- Runs Express on Lambda without code changes.
- Keeps the code consistent between local and cloud environments.

What we considered:

- **Custom Lambda handler:** Would require rewriting the entire API.

Database

PostgreSQL
Purpose: Relational database.

Why we chose it:

- Supports TimescaleDB extension for time-series data.
- ACID compliant for vehicle and user data.
- Good performance for complex queries.

What we considered:

- **MySQL:** Doesn't support TimescaleDB as well.

TimescaleDB
Purpose: Time-series extension.

Why we chose it:

- Keeps relational and time-series data in one database.
- Continuous aggregates make dashboard queries fast.
- Full SQL support.

What we considered:

- **InfluxDB:** Better for pure time-series but less SQL support.
- **Amazon Timestream:** Fully managed but less flexible.

PgBouncer
Purpose: Connection pooling.

Why we chose it:

- Handles many connections without putting load on the database.
- Lightweight and easy to configure.

What we considered:

- **RDS Proxy:** AWS managed but more expensive.

Cloud Infrastructure (AWS)

AWS CDK
Purpose: Infrastructure as Code (IaC).

Why we chose it:

- Codifies our entire stack (API Gateway, Lambdas, S3, Cognito, SQS) in TypeScript.
- Prevents manual ClickOps drift and ensures reproducible deployments (cdk deploy).
- Shares the same language environment as our Node.js/Express backend.

What we considered:

- **Terraform:** Requires learning HCL instead of using existing TypeScript skills.
- **Raw CloudFormation/YAML:** Too verbose and lacks native code logic/loops.

AWS CloudFront
Purpose: Global Content Delivery Network (CDN) for frontend assets.

Why we chose it:

- Delivers the React single-page application over HTTPS with minimal latency.
- Restricts direct access to the S3 bucket using Origin Access Control (OAC).

What we considered:

- **S3 Direct Hosting:** Lacks simple native HTTPS enforcement and global edge caching.

AWS SQS
Purpose: Dead-Letter Queue (DLQ) for failed telemetry ingestion.

Why we chose it:

- Automatically captures failed Kinesis ingestion events without throwing away data.
- Allows batch re-processing after resolving database connectivity issues.

Flyway
Purpose: Database version control and schema migrations.

Why we chose it:

- Automates PostgreSQL and TimescaleDB DDL changes across local Docker and EC2 environments.
- Keeps database schema history tracked in source control.

What we considered:

- **Manual SQL Execution:** Error-prone and unmanageable across a multi-developer team.

Kinesis
Purpose: Real-time telemetry ingestion.

Why we chose it:

- Handles streaming data well.
- Integrates with Lambda natively.
- At-least-once delivery.

What we considered:

- **Kafka:** More powerful but requires more setup and management.

Lambda
Purpose: Serverless processing.

Why we chose it:

- Scales automatically.
- Pay only for what you use.
- No server management.

What we considered:

- **EC2:** More control but requires server management and always-on costs.

API Gateway
Purpose: HTTP routing and JWT validation.

Why we chose it:

- Built-in JWT validation with Cognito.
- Rate limiting and throttling.
- Managed API with CloudWatch integration.

What we considered:

- **Custom Express server:** Would require managing servers and scaling.

Cognito
Purpose: User authentication.

Why we chose it:

- Managed user auth with JWT tokens.
- Integrates with API Gateway.
- Supports MFA and role-based access.

What we considered:

- **Auth0:** Third-party service, more expensive.
- **Firebase Auth:** Different ecosystem, less AWS integration.

EC2
Purpose: Database hosting.

Why we chose it:

- Full control over database configuration.
- Allows installing TimescaleDB extensions.
- Predictable cost for steady workloads.

What we considered:

- **RDS:** Managed but doesn't support TimescaleDB as well.

S3
Purpose: Data archive.

Why we chose it:

- Cheap storage with 11 nines durability.

- Automatic lifecycle policies.
- Integrates with other AWS services.

What we considered:

- **EBS:** More expensive for long-term storage.

CloudWatch
Purpose: Logging and monitoring.

Why we chose it:

- Native integration with Lambda, API Gateway, and EC2.
- Logs and metrics in one place.
- Alerts for anomalies.

What we considered:

- **DataDog:** More features but expensive.
- **Prometheus:** Open-source but requires setup.

DevOps and Testing

Docker
Purpose: Local development.

Why we chose it:

- Works the same on every machine.
- No "works on my machine" issues.
- Easy for new developers to set up.

What we considered:

- **Podman:** Similar but less mature ecosystem.

GitHub Actions
Purpose: CI/CD pipeline.

Why we chose it:

- Runs tests on every pull request automatically.
- Built into GitHub.
- Free for public repositories.

What we considered:

- **Jenkins:** More powerful but requires setup and maintenance.
- **CircleCI:** Paid for private repos.

Jest
Purpose: Testing.

Why we chose it:

- Works for both frontend and backend.
- Built-in coverage reports.
- Fast parallel test execution.

What we considered:

- **Mocha:** More flexible but requires more setup.
- **Vitest:** Newer but less mature ecosystem.

Supertest
Purpose: API testing.

Why we chose it:

- Simulates HTTP requests without a real server.
- Works with Express.
- Good for testing API endpoints.

What we considered:

- **Postman:** Manual testing tool, not suitable for CI/CD.

Codecov
Purpose: Coverage tracking.

Why we chose it:

- Shows coverage trends over time.
- Blocks PRs if coverage drops below 80%.
- Integrated with GitHub.

What we considered:

- **Coveralls:** Similar but less integration with GitHub Actions.

Cypress
Purpose: End-to-end testing.

Why we chose it:

- Tests real user flows in a browser.
- Faster and easier than Selenium.
- Good debugging tools.

What we considered:

- **Selenium:** Slower and harder to set up.
- **Playwright:** Newer but less mature ecosystem.

8. Architecture Design Tactics

Circuit Breaker (Dead-Letter Queue)

- **Mechanism:** Ingestion Lambda → SQS DLQ (telemetry-ingestion-dlq).
- **Quality Attribute:** Availability & Data Integrity
- **Why we chose it:** Captures malformed telemetry payloads or unhandled database connectivity exceptions without stalling Kinesis stream shards or dropping data. Allows offline debugging and message replaying.

Connection Pooling

- **Mechanism:** PgBouncer proxy layer in front of PostgreSQL/TimescaleDB.
- **Quality Attribute:** Availability & Reliability
- **Why we chose it:** Manages database connection limits gracefully during sudden spikes in serverless Lambda concurrency, preventing database resource exhaustion.

Evolutionary Database Migrations

- **Mechanism:** Flyway migration scripts.
- **Quality Attribute:** Maintainability & Testability
- **Why we chose it:** Automates PostgreSQL and TimescaleDB DDL schema changes across local Docker and EC2 environments, ensuring database schema parity across all development and production environments.

Perimeter Token Validation

- **Mechanism:** Cognito User Pool + API Gateway Authorizer.
- **Quality Attribute:** Security & Confidentiality

- **Why we chose it:** Validates short lived JWT signatures at the API Gateway perimeter before requests ever reach the Lambda compute tier, protecting backend resources from unauthorized traffic.

Asynchronous Serverless Compute

- **Mechanism:** AWS Lambda functions for ingestion and API handling.
- **Quality Attribute:** Scalability & Cost Efficiency
- **Why we chose it:** Scales compute capacity instantly from zero to hundreds of concurrent executions in response to incoming traffic, eliminating idle server costs during off-peak hours.

9. Mapping Quality Requirements to Architectural Decisions

Quality Requirement	Architectural Decision	Justification
Scalability	* Amazon Kinesis Data Streams for high-throughput horizontal ingestion scaling. * Concurrent auto-scaling of serverless AWS Lambdas. * TimescaleDB Hypertables (raw_telemetry, clean_telemetry, vehicle_events) partitioned automatically by time intervals.	* Kinesis acts as an elastic buffer that absorbs high-frequency telemetry spikes. * AWS Lambda naturally scales concurrent execution counts up and down to match ingestion volume. * TimescaleDB hypertables prevent performance degradation associated with massive B-Tree index sizes by isolating inserts into specific time-based chunks, maintaining fast, linear

		write speeds as active fleet data scales
Reliability & Fault Tolerance	* AWS Lambda SQS Dead-Letter Queue integration coupled with a database telemetry_errors capture layer. * Flyway Chronological Migrations & Split-Directory Reverse Scripting (paired V and U scripts)	* Isolates malformed JSON payloads at the compute layer, while database processing exceptions capture raw payloads natively alongside a synced_to_cloudwatch tracking mechanism to eliminate data loss. * Flyway schema tables enforce an immutable structural timeline. Split up/down files allow automated verification and deterministic rollback execution (undo) without corrupting production PostGIS or time-series partitions

Performance	* TimescaleDB Continuous Aggregates utilizing background refresh policies * PostGIS Native Spatial Indexing (GIST) paired with C-optimized geometry functions (ST_ClusterDBSCAN, ST_Contains, ST_Intersects). * Partial Unique Indexing for state tracking. * Keyset Pagination (p_before_start_time) for data retrieval. * Sliding-window Telemetry Snapping Views	* Asynchronously pre-computes operational metrics outside the ingestion pipeline, keeping read workflows ultra-fast without loading disk memory during writes. * Offloads heavy GIS math, geofence evaluations, and stop/event clustering to native spatial database indexes (R-Tree), entirely removing slow, resource-heavy in-memory processing loops from application code. * Partial indexes target exclusively active ('open') trip rows, keeping operational indexes exceptionally small and O(1) efficient. * Keyset pagination bypasses costly standard SQL OFFSET scans, guaranteeing identical

		query speeds for deep page requests regardless of total table growth. * Binds real-time fleet asset tracking lookups to a time window, preserving system performance during active map interaction.
Security & Privacy	* AWS Cognito User Pools for secure JWT generation and Role-Based Access Control (RBAC). * TLS 1.2 enforcement and strict rate-limiting traffic policies configured on AWS API Gateway.	* Offloads user sign-in, session tokens, and role evaluation to a dedicated provider. Token verification happens at the API Gateway edge via native authorizers, dropping unauthenticated or rate-limited requests before they ever consume downstream database or compute resources..
Observability	* AWS CloudWatch custom dashboards and logging groups attached to Lambda handlers, integrated directly with an explicit telemetry_errors.synced_to_cloudwatch view flag.	* Provides end-to-end trace visibility. Ingestion schema parse failures retain their full raw payload inside the database while toggling an observability flag, allowing cloud log routers to surface pipeline edge cases

		without destabilizing the application.
Maintainability	* Infrastructure as Code (IaC) implementation via AWS CDK for pipeline deployments, paired with local Docker Compose replication. * Database schema version control managed through Flyway split-directory structures (/migrations and /reverse_migrations).	* AWS CDK ensures repeatable infrastructure definitions and minimizes manual deployment variations. Docker Compose mirrors target services (PostgreSQL/TimescaleDB/PostGIS) locally for instant verification. * Organizing Flyway configurations to process distinct up (V) and down (U) directories eradicates environment drift while safely decoupling ongoing architecture modifications from recovery rollbacks .

10.Deployment Requirements

Rollback Strategies

When changes are deployed to production, the platform protects system stability across all three primary architectural layers:

1. Infrastructure Layer Rollback (AWS CDK / CloudFormation)

Transactional Deployments: Because all physical components are deployed via AWS CDK, the platform inherits atomic deployment guarantees. If an environment deployment encounters an error midway, the entire transaction aborts automatically.

Automatic State Reversion: The deployment framework automatically tears down partially provisioned resources and restores the infrastructure layer to its last known stable configuration.

No-Rollback Debug Mode: For sandbox validation environments, automated rollbacks can be explicitly suspended via CLI parameters, pausing the infrastructure mid-error to let developers inspect configuration problems directly.

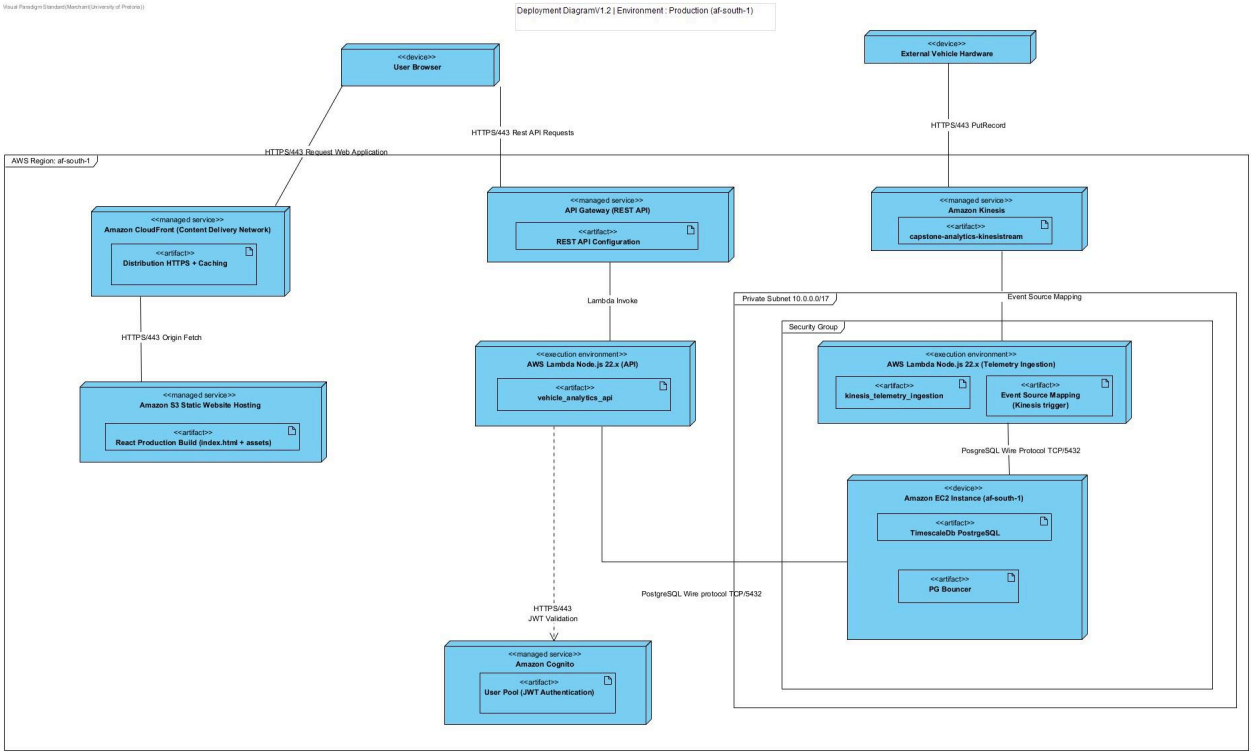
2. Application & Compute Layer Rollback (AWS Lambda)

- **Managed Traffic Shifting:** Updates to API or ingestion compute layers use canary deployment strategies via traffic routing utilities. Instead of migrating all active vehicle traffic instantly, updates are introduced gradually (e.g., routing 10% of traffic to the new version for a 10-minute validation window).
- **Automated Error Alarms:** If system monitoring metrics flag a rise in execution errors or latency spikes that breach our NFR limits during the canary window, the deployment system initiates an immediate rollback, redirecting 100% of live traffic back to the prior stable release version.

3. Database Layer Rollback (PostgreSQL / TimescaleDB Schema)

- **Two-Phase Database Migrations:** Structural data changes are executed via an expand and contract model. Table modifications are split so that the database structure remains fully backward-compatible with the active application code until the new tier stabilizes.
- **Deterministic Schema Rollbacks:** Version changes use Flyway's dual-directory layout.

11.Deployment Diagram



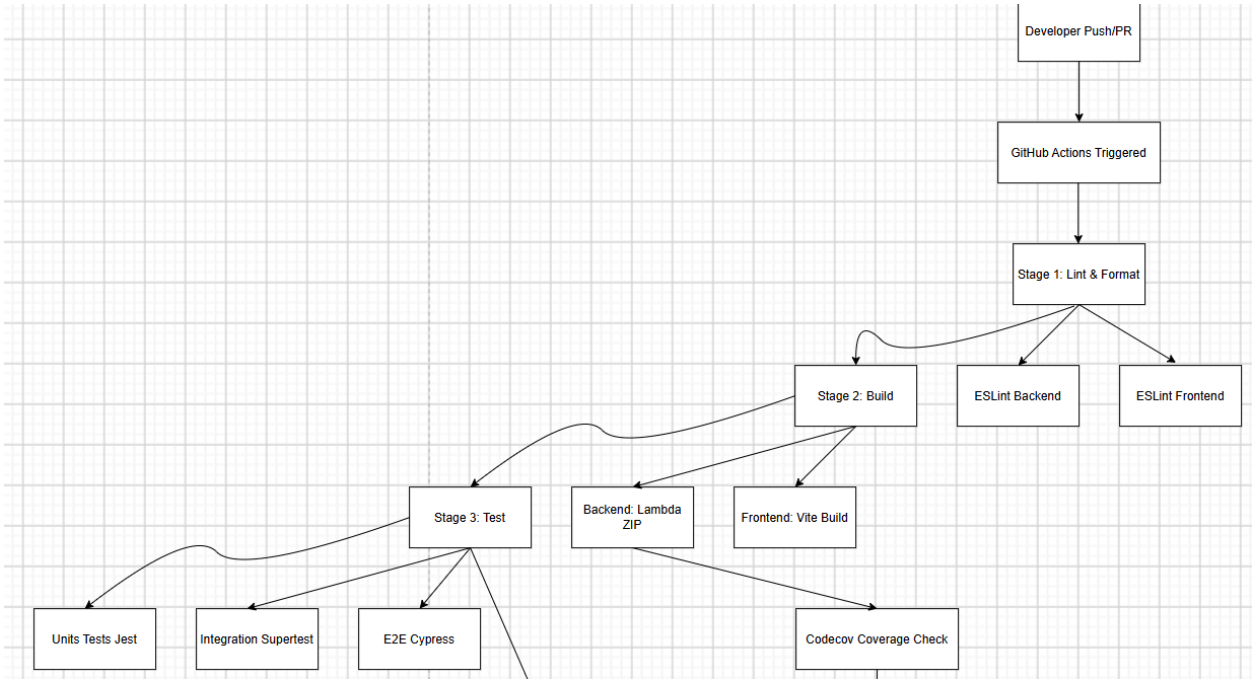
12. CI/CD

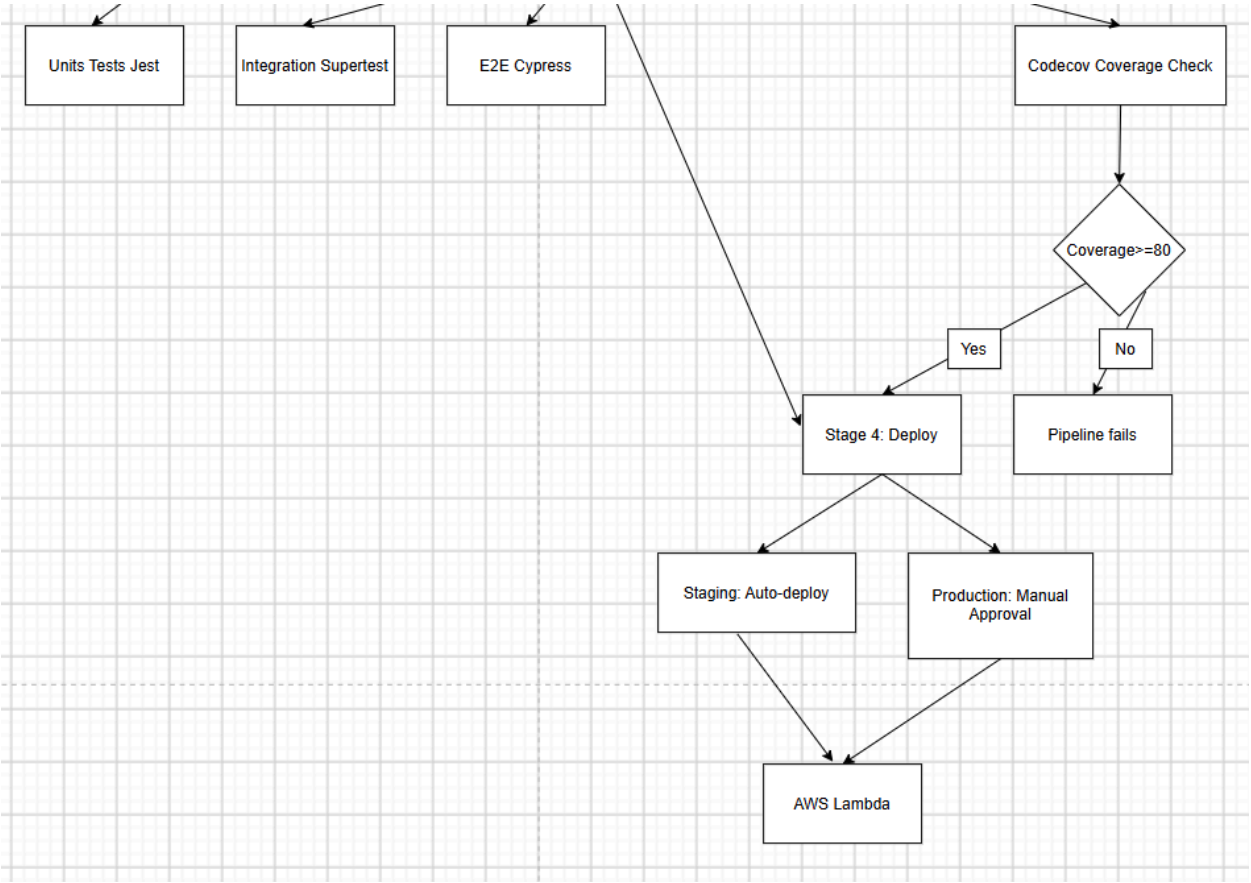
CI/CD Pipeline

Overview

The CI/CD pipeline automates building,testing,and deploying the Vehicle Analytics Platform.Every pull request triggers checks before code can be merged

Pipeline Flow





Pipeline Stages

Lint

Runs ESLint on backend and frontend code. Catches style issues early.Pipeline stops if linting fails

Build

Compiles TypeScript, bundles frontend assets, and packages the Lambda deployment zip.Produces backend_lambda.zip and frontend static files

Test

Runs unit tests (Jest),integration tests (Supertest),and E2E tests (Cypress).Requires 80% coverage.Pipeline stops if any tests fail or coverage drops

Deploy

Deploys to AWS Lambda.Staging deploys automatically on merge to develop.Production requires manual approval (4 reviews) before deploying to main

Branch Strategy

main - Production code. Protected.PR only with 4 approvals

develop - Integration branch. Protected.PR only with 4 approvals

feature/ - Feature development.PR to develop when ready

Environments

Development - Local Docker Compose.No approval needed

Staging - AWS Lambda.Auto-deploys when code merges to develop.Used for testing

Production - AWS Lambda.Manual approval required (4 reviews).Real user traffic

Quality Gates

- Lint: Zero ESLint errors
- Build: Must compile successfully
- Tests: All passing with 80% coverage
- Code Review: 4 approvals required

Rollback

If deployment fails,rollback to previous Lambda version:

```
aws lambda update-function-code \
 --function-name capstone-analytics-backend \
 --zip-file fileb://previous_lambda.zip \
 --region af-south-1
```